Week 6

PyTorch Programming; Autograd and Neural Network Foundations

NMTAFE - Cert IV Data Science and AI

Learning Objectives

- Understand automatic differentiation and autograd in PyTorch
- Construct basic neural network modules and define model architectures
- Trace computation graphs from model input to output and loss
- Apply core concepts to real-world workflows and industry standards

Agenda

- Recap; PyTorch Tensors and Data Structures
- Introduction to Autograd; Automatic Differentiation in PyTorch
- Computation Graphs; How PyTorch Traces Calculations
- Defining Neural Networks in PyTorch
- Practical Exercise; Building and Training a Simple Model
- Industry Case Study; Model Design in a Real AI Pipeline
- Assessment Prep; Key Skills for Upcoming Project

Recap; Tensors and PyTorch Essentials

- PyTorch tensors; Core data structure for all computations
- Previous week covered; Creating, manipulating, and using tensors for ML data workflows
- All deep learning models in PyTorch use tensors under-the-hood

Autograd in PyTorch; Introduction

- Autograd handles automatic differentiation; essential for training neural networks
- Tracks all operations on tensors with `requires_grad=True`
- Automatically builds a dynamic computation graph as code runs

Why Automatic Differentiation Matters

- Gradient computation is core for all ML training; enables model optimization
- Manual derivative calculation is impractical for large models
- PyTorch autograd simplifies gradient computation with minimal code

How Autograd Works; The Core Idea

- Every operation involving tensors creates nodes in a computation graph
- At backward pass, autograd traverses this graph to compute all required gradients
- Computation graph is dynamic; adapts at each model run

Computation Graph Example

```
import torch
x = torch.ones(3, requires_grad=True)
y = x + 2
z = y * y * 3
out = z.mean()
out.backward()
print(x.grad)
```

- Each step builds a node in the computation graph; gradients available after backward
- `x.grad` contains gradients with respect to `x` for use in optimization

Neural Network Modules in PyTorch

- Neural networks are composed of layers; each layer = a module
- PyTorch provides `torch.nn.Module` for defining custom models
- Encapsulates layers, parameters, and the forward computation

Simple Neural Network Example

```
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(10, 5)
        self.fc2 = nn.Linear(5, 1)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return self.fc2(x)
```

- `__init__` defines network architecture; `forward` method describes computation flow

Model Flow; From Input to Prediction

- Data passes through model layers in defined sequence
- Each layer is an operation in computation graph; autograd hooks attach automatically
- Prediction generated; loss calculated; backward pass computes gradients

Practical Exercise; Build and Train a Tiny Model

- Define a small feedforward network for a regression problem
- Use PyTorch's autograd to compute gradients and optimise loss
- Example steps;
  - Create sample input and output tensors
  - Instantiate your network module
  - Use MSELoss and SGD optimizer
  - Perform a forward pass, compute loss, call `backward`, update weights

Activity Instructions

1. Create a new Python file or notebook on your VM or cloud instance
2. Copy the SimpleNet example; modify input and output shapes as needed
3. Generate fake input and output data using `torch.randn`
4. Implement the training loop; print loss at each iteration
5. Experiment; Observe how gradients and outputs change during training

Industry Context; Why Autograd & NN Modules Matter

- Industry models must be trainable and auditable; autograd enables rapid prototyping and debugging
- Neural network modules encourage reusable, configurable code; aligns with MLOps standards and documentation practices
- Modern AI workflows (e.g. Hugging Face, Azure ML) use modular networks for scalable training and deployment

Real-World Scenario; Model Design in Practice

- A local government partner needs a model to predict equipment failure from sensor inputs
- Engineers use PyTorch to prototype; autograd speeds up experimentation, neural modules make design iteratable
- Model deployed to Azure GPU VM; monitored using industry best-practices; adjustments made via custom modules

Assessment Preparation; Key Competencies

- Demonstrate use of autograd in PyTorch for gradient calculation
- Implement and train a basic neural network using nn.Module
- Explain the computation graph behind a forward and backward pass
- Review; Practical exercises and today’s activity can form part of your portfolio project

Key Takeaways

- Autograd handles automatic differentiation; essential for modern ML development
- Neural network modules enable structured and maintainable model code
- These concepts are core for upcoming assessments and real AI engineering roles

Links to Prior Weeks

- Week 5; PyTorch tensor operations are foundational to autograd and neural networks
- Week 4; Good coding standards and version control help manage modular PyTorch projects
- Week 3; Scripting and pseudocode skills aid in designing robust model training pipelines

Next Steps

- Complete today’s exercises and review example notebooks
- Start thinking about possible topics and architectures for your PyTorch Fundamentals Project
- Reach out if you need help with code, concepts, or cloud setups; contact details in the LMS

Questions and Discussion

- What aspect of autograd or neural modules is most confusing or intriguing?
- How might you use these skills in your own AI projects or future role?
- Let’s explore code or issues together before the next session